

MODULE 1

Software Engineering-An Introduction

Software Engineering-Definition, Software life cycle activities, Challenges in System Development, Software process Models: Waterfall, Prototyping, Evolutionary, Spiral, Unified and Agile Model, Software Development Methodology.

1.1 INTRODUCTION TO SOFTWARE ENGINEERING

Software is a program or set of programs containing instructions that provide the desired functionality. Engineering is the process of designing and building something that serves a particular purpose and finds a cost-effective solution to problems.

Process:

A process is the sequence of steps executed to achieve a goal. A process is defined by cycles. Similar to a project, a process also has a beginning, middle, and end; however, this cycle repeats itself over an average period of time.

Project:

A project is defined by a fixed time, scope, and resources. When implementing a project, the goal is to execute change, usually drastic, and to incorporate that change into the day-to-day processes of the company.

Both projects and process are important for running and improving a business. However, depending on the end result trying to be achieved, one may be more suited than the other.

What is Software Engineering?

Software Engineering is the process of designing, developing, testing, and maintaining software. It is a systematic and disciplined approach to software development that aims to create high-quality, reliable, and maintainable software.

1. Software engineering includes a variety of techniques, tools, and methodologies, including requirements analysis, design, testing, and maintenance.
2. It is a rapidly evolving field, and new tools and technologies are constantly being developed to improve the software development process.
3. By following the principles of software engineering and using the appropriate tools and methodologies, software developers can create high-quality, reliable, and maintainable software that meets the needs of its users.
4. Software Engineering is mainly used for large projects based on software systems rather than single programs or applications.
5. The main goal of Software Engineering is to develop software applications for improving quality, budget, and time efficiency.
6. Software Engineering ensures that the software that has to be built should be consistent, correct, also on budget, on time, and within the required requirements.

Key Principles of Software Engineering

1. Modularity: Breaking the software into smaller, reusable components that can be developed and tested independently.
2. Abstraction: Hiding the implementation details of a component and exposing only the necessary functionality to other parts of the software.
3. Encapsulation: Wrapping up the data and functions of an object into a single unit, and protecting the internal state of an object from external modifications.
4. Reusability: Creating components that can be used in multiple projects, which can save time and resources.

5. Maintenance: Regularly updating and improving the software to fix bugs, add new features, and address security vulnerabilities.
6. Testing: Verifying that the software meets its requirements and is free of bugs.
7. Design Patterns: Solving recurring problems in software design by providing templates for solving them.
8. Agile methodologies: Using iterative and incremental development processes that focus on customer satisfaction, rapid delivery, and flexibility.
9. Continuous Integration & Deployment: Continuously integrating the code changes and deploying them into the production environment.

Main Attributes of Software Engineering

Software Engineering is a systematic, disciplined, quantifiable study and approach to the design, development, operation, and maintenance of a software system. There are four main Attributes of Software Engineering.

1. Efficiency: It provides a measure of the resource requirement of a software product efficiently.
2. Reliability: It assures that the product will deliver the same results when used in similar working environment.
3. Reusability: This attribute makes sure that the module can be used in multiple applications.
4. Maintainability: It is the ability of the software to be modified, repaired, or enhanced easily with changing requirements.

Dual Role of Software

There is a dual role of software in the industry. The first one is as a product and the other one is as a vehicle for delivering the product. We will discuss both of them.

1. As a Product

It delivers computing potential across networks of Hardware.

It enables the Hardware to deliver the expected functionality.

Example:

- MS Word is a software product used by people to create documents.
- Mobile banking app helps customers manage their accounts.

It acts as an information transformer because it produces, manages, acquires, modifies, displays, or transmits information.

2. As a Vehicle for Delivering a Product

It provides system functionality (e.g., payroll system).

It controls other software (e.g., an operating system).

It helps build other software (e.g., software tools).

Example:

- The firmware in a washing machine controls the hardware and delivers the washing functionality.
- Air traffic control software is not the end product itself but the medium through which the safety service is delivered.

Objectives of Software Engineering

Maintainability: It should be feasible for the software to evolve to meet changing requirements.

Efficiency: The software should not make wasteful use of computing devices such as memory, processor cycles, etc.

Correctness: A software product is correct if the different requirements specified in the SRS Document have been correctly implemented.

Reusability: A software product has good reusability if the different modules of the product can easily be reused to develop new products.

Testability: Here software facilitates both the establishment of test criteria and the evaluation of the software concerning those criteria.

Reliability: It is an attribute of software quality. The extent to which a program can be expected to perform its desired function, over an arbitrary time period.

Portability: In this case, the software can be transferred from one computer system or environment to another.

Adaptability: In this case, the software allows differing system constraints and the user needs to be satisfied by making changes to the software.

Interoperability: Capability of 2 or more functional units to process data cooperatively.

What Careers Are There in Software Engineering?

A degree in software engineering and relevant experience can be utilized to explore several computing job choices. Software engineers have the opportunity to seek well-paying careers and professional progress, although their exact possibilities may vary depending on their particular school, industry, and region.

Following are the job choices in software engineering:

- SWE (Software Engineer)
- SDE (Software Development Engineer)
- Web Developer
- Quality Assurance Engineer
- Web Designer
- Software Test Engineer
- Cloud Engineer ·
- Front-End Developer
- Back-End Developer
- DevOps Engineer.
- Security Engineer.

What Tasks do Software Engineers do?

The main responsibility of a software engineer is to develop useful computer programs and applications. Working in teams, you would complete various projects and develop solutions to satisfy certain customer or corporate demands.

Some of the key responsibilities of software engineer are:

- **Requirement Analysis:** Collaborating with stakeholders to understand and gather the requirements to design and develop software solutions.
- **Design and Development:** Creating well-structured, maintainable code that meets the functional requirements and adheres to software design principles.
- **Testing and Debugging:** Writing and conducting unit tests, integration tests, and debugging code to ensure software is reliable and bug-free.
- **Code Review:** Participating in code reviews to improve code quality, ensure adherence to standards, and facilitate knowledge sharing among team members.
- **Maintenance:** Updating and maintaining existing software systems, fixing bugs, and improving performance or adding new features.
- **Documentation:** Writing clear documentation, including code comments, API documentation, and design documents to help other engineers and future developers understand the system.

How Much do Software Engineers Make?

The majority of software developers globally make less than \$50,000 annually on average. This covers not only junior developers but also some senior ones based on where they work and for what company. However, it is more likely that a mid-level or senior software engineer makes between \$50,000 and \$120,000 per year.

What Skills do Software Engineers Need?

To achieve success, software engineers require a unique set of technical and soft skills. These skills include computer programming knowledge for designing, developing, testing, and

debugging software, as well as soft skills for interacting with stakeholders, team members, and company leadership.

1.2 SOFTWARE DEVELOPMENT LIFE CYCLE (SDLC):

Software Development Life Cycle, SDLC for short, is a well-defined, structured sequence of stages in software engineering to develop the intended software product.

SDLC Activities: SDLC provides a series of steps to be followed to design and develop a software product efficiently. SDLC framework includes the following steps:

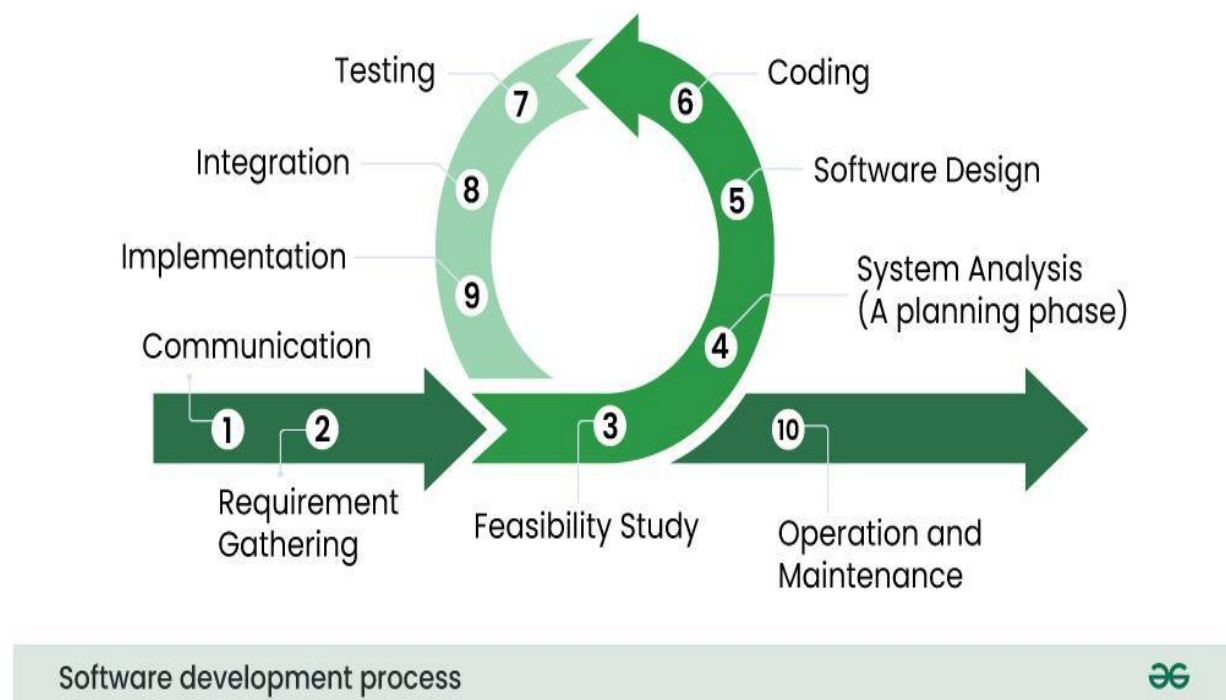


Figure1.2: Software Development life Cycle.

Communication

- This is the first step where the user initiates the request for a desired software product. He contacts the service provider and tries to negotiate the terms. He submits his request to the service providing organization in writing.

Requirement Gathering

- This step onwards the software development team works to carry on the project. The team holds discussions with various stakeholders from problem domain and tries to bring out as much information as possible on their requirements.
- The requirements are contemplated and segregated into user requirements, system requirements and functional requirements.
- The requirements are collected using a number of practices as given: studying the existing or obsolete system and software, conducting interviews of users and developers, referring to the database or collecting answers from the questionnaires.

Feasibility Study

- After requirement gathering, the team comes up with a rough plan of software process.
- At this step the team analyzes if a software can be made to fulfill all requirements of the user and if there is any possibility of software being no more useful.
- It is found out, if the project is financially, practically and technologically feasible for the organization to take up. There are many algorithms available, which help the developers to conclude the feasibility of a software project.

System Analysis

- At this step the developers decide a roadmap of their plan and try to bring up the best software model suitable for the project.
- System analysis includes Understanding of software product limitations, learning system related problems or changes to be done in existing systems beforehand, identifying and addressing the impact of project on organization and personnel etc.
- The project team analyzes the scope of the project and plans the schedule and resources accordingly.

Software Design

- Next step is to bring down whole knowledge of requirements and analysis on the desk and design the software product.

- The inputs from users and information gathered in requirement gathering phase are the inputs of this step.
- The output of this step comes in the form of two designs; logical design and physical design. Engineers produce meta-data and data dictionaries, logical diagrams, data-flow diagrams and in some cases pseudo codes.

Coding

- This step is also known as programming phase. The implementation of software design starts in terms of writing program code in the suitable programming language and developing error-free executable programs efficiently.

Testing

- An estimate says that 50% of whole software development process should be tested. Errors may ruin the software from critical level to its own removal.
- Software testing is done while coding by the developers and thorough testing is conducted by testing experts at various levels of code such as module testing, program testing, product testing, in-house testing and testing the product at users end.
- Early discovery of errors and their remedy is the key to reliable software.

Integration

- Software may need to be integrated with the libraries, databases and other program(s). This stage of SDLC is involved in the integration of software with outer world entities.

Implementation

- This means installing the software on user machines. At times, software needs post-installation configurations at user end. Software is tested for portability and adaptability and integration related issues are solved during implementation.

Operation and Maintenance

- This phase confirms the software operation in terms of more efficiency and less errors. If required, the users are trained on, or aided with the documentation on how to operate the software and how to keep the software operational.
- The software is maintained timely by updating the code according to the changes taking place in user end environment or technology. This phase may face challenges from hidden bugs and real-world unidentified problems.

Disposition

- As time elapses, the software may decline on the performance front. It may go completely obsolete or may need intense upgradation. Hence a pressing need to eliminate a major portion of the system arises.
- This phase includes archiving data and required software components, closing down the system, planning disposition activity and terminating system at appropriate end-of-system time.

1.3 CHALLENGES IN SYSTEM DEVELOPMENT

Key Takeaways:

On a software development journey, you may trip over a number of obstacles that could hinder your final outcomes. Being able to anticipate them can equip you to either sidestep them or brace for any potential difficulties later on.

- **Insufficient Direction and Leadership:** A clear vision and guidance from project leaders/managers are crucial for steering development efforts on the right path, and vice versa.
- **Difficulties in Estimating Time and Resources Accurately:** It is usually a daunting, risky task to gauge the project scope and allocate resources accordingly. If not done properly, project delays and budget overruns are inevitable.
- **The Scarcity of Skilled Developers:** The demand for experienced personnel often surpasses supply. As a result, lots of projects find it costly and challenging to find and hire tech talent.

- **Project Scope Creep/Requirement Creep:** Ambiguity in clarifying project goals, scope, and requirements may derail the development efforts or jeopardize the entire SDLC.
- **Communication Barriers:** Any breakdown in communication may result in misunderstanding and errors, negatively affecting the project's progression.
- **Rigid Deadlines:** Tight schedules not only put immense pressure on development teams but also compromise the software quality.
- **Testing and Debugging:** This is a time-consuming and effort-intensive process that is critical yet hazardous.
- **Maintaining Competitiveness:** The continuously evolving tech landscapes demand that development teams stay up-to-date with the latest technologies and adapt to maintain an edge.
- **Opting for Development Methodologies:** The development approach that does not align with the project's needs may hinder progress and outcomes.

Some Examples Project development Challenge:

Project Reality 1: Many system development projects have long development durations, which typically range from one year to several years. Real-world projects must meet schedule and budget constraints.

Project Challenge 1: How do we plan, schedule, and manage the project work without a complete knowledge of what the customer wants, and what will happen in the next several years?

Project Reality 2: Many system development projects require collaboration of multiple departments or development teams. For example, many large embedded systems involve both hardware and software components and require the software engineering department to cooperate with hardware departments.

Project Challenge 2: How do we divide the work and assign the interdependent pieces to the departments and teams, and be able to smoothly integrate the components produced by the different departments and teams?

Project Reality 3: Different departments or teams may use different development processes, methods, and tools. The departments or teams may be located at different places.

Project Challenge 3: How do we ensure proper communication and coordination among the departments and teams?

Besides project challenges, there are product challenges:

System Reality 1: Many real-world systems have to satisfy a large number of requirements and constraints.

System Challenge 1: How do we develop the system to ensure that the requirements and constraints are met?

System Reality 2: Requirements and constraints may change from time to time. Take for example a small project that the author had contracted to do. The requirements had to change every week during the first three months due to change requests from the customer.

System Challenge 2: How do we design the processes and the products to cope with changes that are often inevitable and the exact changes are impossible to predict?

System Reality 3. The system may consist of hardware and software components, use third party components and multiple implementation languages, and run on multiple platforms and machines located at different places.

System Challenge 3. How do we design the system so that changes can be made relatively easily and without much impact to the rest of the system?

Absence of Guidance from Management

- Software developers need clear instructions and guidance from project managers in order to handle their jobs effectively. Therefore, if they are left in the dark, it can result in chaos and confusion, which affects the overall software product quality in one way or another. This is the most common yet biggest challenge that possibly leads to project failure or delays.
- Without proper strategy or direction, you and other team members could be miles away from the right track. Thus, let's trace to the root, which is as obvious as daylight

- to establish a clear and concise project plan. The project manager communicates and collaborates with all stakeholders (Including business analysts, quality assurance engineers, technical architects, developers, etc.)

- Together, they work to define the software's objectives, scope, timeline, and milestones. In this way, all involved stay on the same page. Beyond planning, management positions must actively engage with each team member to foster a work environment that is conducive to productivity.

Difficulty in Estimating Time & Resources

- It is not rare to see software development projects facing the issues of estimating resources, which later leads to insufficient allocation.
- Given the time and budget constraints, it is important to have realistic timelines for a software project that outlines every single detail, stage, and task of the project. By visualizing the whole roadmap, project managers or leaders have better insights to calculate and balance the workload and manpower in accordance with the defined timeline.
- This way, developers know how much time they can spend on enhancing a certain feature or working on software testing and bug fixes, for instance. If a developer has too little time to work on the part of the project, it may not function properly or have features that are lacking. If we underestimate the amount of time it takes to complete a task, there will be delays in the project, and other members of the team may run out of work. On the other hand, if it takes longer than expected, then the project will stretch into an uncomfortable timeframe.
- In terms of solutions, project managers should also consider one way - to break larger tasks into smaller chunks so they are more manageable. Also, developers can make use of project management tools (Such as Toggl or Harvest) to keep track of how much effort their teams are investing in a single task.

Lack of Resources for the Software Development Process

- Software product development requires a variety of resources, including advanced technologies, tools, and expertise. Oftentimes, the challenge just mentioned above

results in the insufficiency of resources when the SDLC (Software Development Life Cycle) actually occurs. It could be the shortage of skilled developers, budgets, tech stack, or tools. Overall, these constraints may hinder the progress of a project or even prevent it from being completed.

- Part of the hurdle is due to the increasingly complex project requirements in an ever-changing technology ecosystem. In such a situation, the project team may not have enough funds to hire one new software developer or several who specialize in the required tech stack (Programming languages, frameworks, or libraries). Also, the lack of sufficient team members means a growing workload is placed on the shoulders of the existing teams. If not addressed properly, this issue will escalate and lead to software engineer burnout and further decreased productivity. What's more, acquiring licenses for certain software or communication tools could also add to the financial burden.
- So, what can be done? Fortunately, there are multiple workarounds for these road-blockers. For human resources, project leaders have a good option - to contract out a dedicated team or hire freelance developers to work alongside as an extension of their in-house team. When you opt for outsourcing services, you can save costs while still securing quality work. In terms of tech stack, investing in cross-training and upskilling programs for existing team members is a viable solution, especially when you are short on budget.
- If developers do not have the right tools, it will be impossible for them to meet deadlines or produce a high-quality product. Since many different types of assets are available on the internet – from fonts and graphics to code libraries and scripts – it's possible to find something that addresses your specific needs without having to spend anything at all. Managers can seek out and leverage open-source platforms, software tools, and libraries that are free of charge or budget-friendly. Obviously, it is a pretty good idea to ease the budgetary burden of one project.

Defining the Requirements of the Software Projects

- One of the most time-consuming tasks before the development phase is to map out the project scope and requirements. Simply, this means figuring out what the product

should do, how it should work, what problems it should solve, which features are needed, and how the various aspects of the system interact.

- The requirements will shape the very first structures of any software application. Therefore, the detailed descriptions of what is expected to be the end project delivery must be straightforward, concise, and comprehensive. As such, software engineers have a good understanding of what they need to do. Unfortunately, it's often difficult to get a clear picture of what's required and even more difficult to translate that into specific instructions. If these requirements are not well-defined, it can lead to confusion and frustration on the part of the developers, which will ultimately delay the project.
- It is essential to have clear software development initiatives and an exemplary process for defining the final agreed-upon requirements. This way, the development teams will know at a glance what they need to do, and their work won't get unnecessarily stalled. One useful way to address this problem is by holding discussions with customers or clients when there are any questions or unclear requirements during the project cycle. Having these discussions will allow for greater clarity on what needs to be done and when things need to be completed. Moreover, the project team should create prototypes and test them with the customers for feedback.

Communication Breakdowns

- Proper communication is the ultimate key to project success, which is not an easy goal to accomplish. Yet, it is not impossible. Miscommunication among stakeholders is a major concern during the project development phase. Why bother? These are common challenges, but still, they may result in a chain of consequences. Tangibly, the failure to communicate efficiently may cause confusion, misunderstanding, dissatisfaction, or even conflicts between team members.
- To prevent these challenges of software development from occurring, you should invest in building an open, seamless line of communication between all parties involved: Establish an efficient communication process (with regular meetings or daily updates), employ new tools (like messaging apps, video conferencing tools or

project management software), and use best practices (i.e., clear and concise communication, documenting everything that happens during the project). In addition, developers should be more proactive in connecting and exchanging info with others - asking questions, seeking clarification whenever there is any doubt, and giving timely feedback.

Under Strict Time Constraints:



- Next, developers need to be ready to deal with strict time constraints. In a few cases, deadlines are agreed upon even before they know what work needs to be done. In many cases, deadlines are agreed upon before they even know what work needs to be done, leading to a lot of pressure and frustration. Even if this issue doesn't happen initially, it will most likely arise later in the project when the software development team encounters problems that might take longer than expected to address.
- To deal with this challenge, companies should put an emphasis on good time management. Developers and project managers should set realistic deadlines and clear expectations from the start. However, they must be ready and willing to make adjustments when necessary because incidents or unexpected changes may occur halfway and make the project diverge from the preset trajectory.
- Moreover, it's advisable for software development teams to take the necessary time off for rest and recovery and build in time for contingencies. This way, even if unexpected events arise or the team finds themselves running behind, they can make up for lost time quickly without putting the project at risk.

Over-complicating the Software Development Project

- Project complexity could be one of the top challenges that induce headaches for developers. In an effort to build robust, full-fledged software products, you and your team may end up wanting too many things and adding unnecessary features or functions that do not align with the actual needs of the targeted users. As a result, developers may get stuck and have no idea where or how to proceed, which makes it troublesome to manage projects in the long run.
- Developers need more than a well-defined plan to avoid such an issue. Instead, they should regularly review and refine the project scope as well as requirements and devise strategies for dealing with false starts. In this way, they are able to prioritize features and functionalities on their importance and relevance to the final product. Meanwhile, they should also break down the project into manageable tasks and deal with potential problems at a micro level.
- Don't forget to stay in close communication with the team members so that everyone is well aware of what is happening and can help out when needed. Accomplish all these, and you will be able to keep the project organized and stay on track with the entire development cycle.

Hunting for Qualified Software Developers:

- In the current IT market, software developers, particularly skilled ones, are challenging to find, hire, and retain. There are many reasons behind this, such as the shortage of qualified workers and the overhead costs of recruiting, not to mention the troubles of getting the wrong hires or the resources and effort to maintain a full-fledged team.



- In addition, a majority of tech companies nowadays are reluctant to employ fresh graduates due to the concern of lacking experience and necessary skills.
- To overcome this challenge, companies need to be more proactive in their search for talent. They should identify the skills they need and target candidates who have those skills. They should also create an attractive job offer that includes benefits and a competitive salary. In addition, they should actively promote their company and its culture. By doing these things, companies can attract more talent and continue to grow their businesses.
- Moreover, companies can also consider using recruitment agencies or online platforms that provide ready-made resumes for professionals with different qualifications and skill sets. Or they should be more flexible in talent acquisition by giving more opportunities to interns. Currently, more and more many software companies are outsourcing software development work to third-party companies, which can be a more cost-effective way to find the right talent.

Testing & Debugging

- In any software development lifecycle, testing is always of vital importance. Unfortunately, many projects have difficulties dealing with this phase. Software testing and debugging usually involve identifying and fixing the errors in the code. In addition, it can sometimes be hard to determine the source of the error and how to fix it. Or developers may be too occupied with other vital tasks, being less attentive to the code review process.
- With complex projects, there is always a risk for bugs and errors to occur, even after thorough testing. This can lead to frustration for both engineers and end-users. A viable solution here is to adopt a test-driven development (TDD) approach - in which tests are written before the code. Developers will focus on writing code that passes the test, hence lowering the chances of bugs and errors. It is advised to have your developers cross-check each other's code on a regular basis to give feedback and catch issues early on. Furthermore, they should use automated debugging tools to help them ease the testing process.

Maintaining the Competitive Edge

- We are proceeding further into the fast-paced and ever-changing technology landscape wherein there are too many companies that offer similar products and services. Thus, it is a must to keep your own set of USPs (Unique Selling Points) and remain relevant in order to survive and thrive.
- Tech companies need to continually innovate and improve their product and service offerings from project to project to stay ahead of the curve. They should also implement a formal process for managing these changes in order to stay on track with the development process. Moreover, they need to keep an eye on the latest market trends and stay up-to-date with new technologies. Furthermore, companies should consider investing in R&D initiatives to identify opportunities for growth. By doing these things, you can continue to evolve with the market and better cater to your customers' needs.

Unfitted Software Development Methodology

- Aside from the aforementioned challenges in software development, opting for unsuitable methodologies can bring about more serious issues in the project.
- Different methods (i.e., Agile, Waterfall, Scrum, etc.) are suited to different types of projects with their unique sets of requirements and demands. Hence, a software development approach can work flawlessly for one project, but there is no guarantee it will do the same for another. The key point is about the best fit for a specific project, not the other way around. Choose wisely.

Navigate Through All with the Aid from Orient Software

- For professional support and solving software development challenges, you should consult with an IT partner like Orient Software.
- For almost two decades in the IT outsourcing industry and still counting, Orient Software has carved its name as the leading software development service provider in the local area as well as worldwide. At Orient Software, we have nurtured a large

team of experts and tech talent with realistic experience in projects of different types and scales. Check out the case studies for typical successful ones.

- We have been collaborating with hundreds of clients (startups, SMBs, and enterprises) from all around the world, so we work with an international style and culture. Our service range is noticeably comprehensive, consisting of custom software development, UI/UX design, QA testing, IT consulting, staff augmentation, and many more.
- We can be either your trusted strategist or your executor of a project. We can also function as your entire dedicated team if needed (i.e., offshore development center). No matter the size or complexity of your project, Orient Software has the expertise and resources to deliver high-quality outcomes. Just tell us what your expectations are. Contact us today, and we will get back to you within only three business days.

1.4 SOFTWARE DEVELOPMENT PARADIGM or SOFTWARE PROCESS MODELS:

- The software development paradigm helps developer to select a strategy to develop the software. A software development paradigm has its own set of tools, methods and procedures, which are expressed clearly and defines software development life cycle.
- Software processes are the activities for designing, implementing, and testing a software system. The software development process is complicated and involves a lot more than technical knowledge.
- That's where software process models come in handy. A software process model is an abstract representation of the development process.
- A software process model is an abstraction of the software development process. The models specify the stages and order of a process. So, think of this as a representation of the order of activities of the process and the sequence in which they are performed.
- There are many kinds of process models for meeting different requirements. We refer to these as SDLC models (Software Development Life Cycle models). The most popular and important SDLC models are as follows:

- **Waterfall model**
- V model
- Incremental model
- RAD model
- **Agile model**
- Iterative model
- **Prototype model**
- **Spiral model**

Factors in choosing a software process:

- Choosing the right software process model for your project can be difficult. If you know your requirements well, it will be easier to select a model that best matches your needs.

You need to keep the following factors in mind when selecting your software process model:

Project requirements

- Before you choose a model, take some time to go through the project requirements and clarify them alongside your organization's or team's expectations.
- Will the user need to specify requirements in detail after each iterative session?
- Will the requirements *change* during the development process?

Project size

- Consider the size of the project you will be working on.
- Larger projects mean bigger teams, so you'll need more extensive and elaborate project management plans.

Project complexity

- Complex projects may not have clear requirements.
- The requirements may change often, and the cost of delay is high. Ask yourself if the project requires constant monitoring or feedback from the client.

Cost of delay

- Is the project highly time-bound with a huge cost of delay, or are the timelines flexible?

Customer involvement

- Do you need to consult the customers during the process? Does the user need to participate in all phases?

Familiarity with technology

- This involves the developers' knowledge and experience with the project domain, software tools, language, and methods needed for development.

Project resources

- This involves the amount and availability of funds, staff, and other resources.

Waterfall Model:

Waterfall model is the simplest model of software development paradigm. It says the all the phases of SDLC will function one after another in linear manner. That is, when the first phase is finished then only the second phase will start and so on.

This model assumes that everything is carried out and taken place perfectly as planned in the previous stage and there is no need to think about the past issues that may arise in the next phase. This model does not work smoothly if there are some issues left at the previous step. The sequential nature of model does not allow us go back and undo or redo our actions. This model is best suited when developers already have designed and developed similar software in the past and are aware of all its domains.

The Waterfall Model has **six phases** which are:

- 1. Requirement Analysis:** The first phase involves gathering requirements from stakeholders and analyzing them to understand the scope and objectives of the project.
- 2. System Design:** Once the requirements are understood, the design phase begins. This involves creating a detailed design document that outlines the software architecture, user interface, and system components.

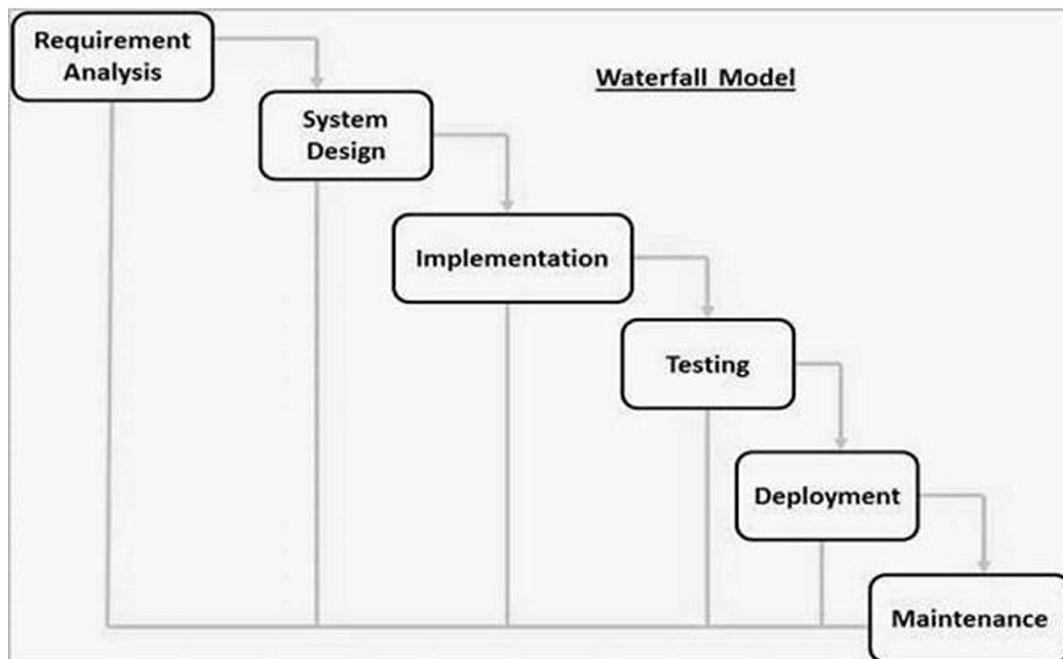


Figure1.4.1: Waterfall Model

3. Development/Implementation: The Development phase include implementation involves coding the software based on the design specifications. This phase also includes unit testing to ensure that each component of the software is working as expected.

4. Testing: In the testing phase, the software is tested as a whole to ensure that it meets the requirements and is free from defects.

5. Deployment: Once the software has been tested and approved, it is deployed to the production environment.

6. Maintenance: The final phase of the Waterfall Model is maintenance, which involves fixing any issues that arise after the software has been deployed and ensuring that it continues to meet the requirements over time.

When to use SDLC Waterfall Model?

Some Circumstances where the use of the Waterfall model is most suited are:

- When the requirements are constant and not changed regularly.
- A project is short

- The situation is calm
- Where the tools and technology used is consistent and is not changing
- When resources are well prepared and are available to use.

Advantages of Waterfall Model

1. **Simple and Easy to Understand:** The sequential phases (requirement → design → implementation → testing → deployment → maintenance) are straightforward to follow. Suitable for beginners and small teams.
2. **Structured Approach:** Each phase has well-defined deliverables and documentation, making it easy to manage.
3. **Easy to Manage:** Since progress flows in a linear order, project tracking and milestone reviews are simpler.
4. **Good for Small Projects:** Works well when requirements are clearly defined and unlikely to change.
5. **Early Documentation:** Complete documentation is created before coding, which helps in knowledge transfer and future maintenance.
6. **Clear Requirement Stability:** If requirements are well-known upfront, the model ensures minimal confusion during later phases.

Disadvantages of Waterfall Model

1. **Inflexible to Changes:** Once a phase is completed, going back is difficult. Changes in requirements during later stages are costly.
2. **Late Testing:** Testing starts only after development is complete, so bugs may be discovered late in the process.
3. **High Risk and Uncertainty:** If initial requirements are misunderstood, the entire project may fail, as feedback is gathered only at the end.

4. **Not Suitable for Complex/Long Projects:** Large, evolving projects require frequent changes, which the waterfall model cannot handle efficiently.
5. **Customer Involvement is Low:** Client feedback is minimal during development, so the final product may not fully meet expectations.
6. **Slow Delivery:** The working product is available only after the last phase, making it unsuitable when quick releases are needed.

Prototyping Model:

Prototyping Model is a way of developing software where an early version, or prototype, of the product is created and shared with users for feedback.

What is Prototyping Model?

The Prototyping Model is one of the most popularly used Software Development Life Cycle Models (SDLC models). This model is used when the customers do not know the exact project requirements beforehand.

In this model, a prototype of the end product is first developed, tested, and refined as per customer feedback repeatedly till a final acceptable prototype is achieved which forms the basis for developing the final product.

Key Features of the Prototyping Model:

Early Feedback: A quick, working model of the software is developed, which allows users and stakeholders to give feedback early in the development process.

Iterative Process: The model evolves through multiple iterations, refining the prototype based on user input, until the desired system is achieved.

Focus on User Interaction: The model prioritizes user interaction and involvement, ensuring that the software aligns closely with user needs and expectations.

Prototyping Model-Concept

In this process model, the system is partially implemented before or during the analysis phase thereby allowing the customers to see the product early in the life cycle. The process starts by interviewing the customers and developing the incomplete high-level paper model.

This document is used to build the initial prototype supporting only the basic functionality as desired by the customer. Once the customer figures out the problems, the prototype is further refined to eliminate them. The process continues until the user approves the prototype and finds the working model to be satisfactory.

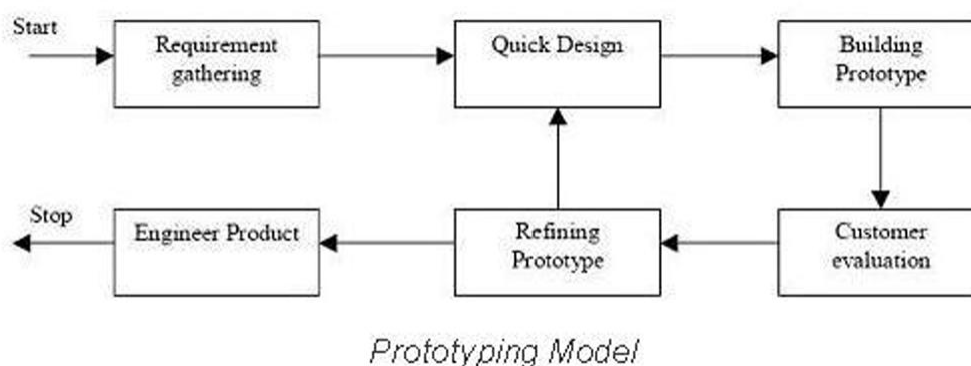


Figure 1.4.2: Prototyping Model

Phases of Prototyping Model

Prototyping Model has six phases as follows: Prototyping Model Phases

1. **Requirements gathering and analysis:** Requirement analysis is the first step in developing a prototyping model. During this phase, the system's desires are precisely defined. During the method, system users are interviewed to determine what they expect from the system.

2. **Quick design:** The second phase could consist of a preliminary design or a quick design. During this stage, the system's basic design is formed. However, it is not a complete design. It provides the user with a quick overview of the system. The rapid design aids in the development of the prototype.

3. Build a Prototype: During this stage, an actual prototype is intended to support the knowledge gained from quick design. It is a small low-level working model of the desired system.

4. Initial user evaluation: The proposed system is presented to the client for preliminary testing at this stage. It is beneficial to investigate the performance model's strengths and weaknesses. Customer feedback and suggestions are gathered and forwarded to the developer.

5. Refining prototype: If the user is dissatisfied with the current model, you may want to improve the type that responds to user feedback and suggestions. When the user is satisfied with the upgraded model, a final system based on the approved final type is created.

6. Implement Product and Maintain: The final system was fully tested and distributed to production after it was developed to support the original version. To reduce downtime and prevent major failures, the programmer is run on a regular basis.

Types of Prototyping Models

There are four types of Prototyping Models, which are described below.

1. Rapid Throwaway Prototyping: This technique offers a useful method of exploring ideas and getting customer feedback for each of them. In this method, a developed prototype need not necessarily be a part of the accepted prototype. Customer feedback helps prevent unnecessary design faults and hence, the final prototype developed is of better quality.

2. Evolutionary Prototyping: In this method, the prototype developed initially is incrementally refined based on customer feedback till it finally gets accepted. In comparison to Rapid Throwaway Prototyping, it offers a better approach that saves time as well as effort. This is because developing a prototype from scratch for every iteration of the process can sometimes be very frustrating for the developers.

3. Incremental Prototyping: In this type of incremental prototyping, the final expected product is broken into different small pieces of prototypes and developed individually. In the end, when all individual pieces are properly developed, then the different prototypes are collectively merged into a single final product in their predefined order. It's a very efficient

approach that reduces the complexity of the development process, where the goal is divided into sub-parts and each sub-part is developed individually.

The time interval between the project's beginning and final delivery is substantially reduced because all parts of the system are prototyped and tested simultaneously. Of course, there might be the possibility that the pieces just do not fit together due to some lack of ness in the development phase – this can only be fixed by careful and complete plotting of the entire system before prototyping starts.

4. Extreme Prototyping: This method is mainly used for web development. It consists of three sequential independent phases:

- In this phase, a basic prototype with all the existing static pages is presented in HTML format.
- In the 2nd phase, Functional screens are made with a simulated data process using a prototype services layer.
- This is the final step where all the services are implemented and associated with the final prototype.

This Extreme Prototyping method makes the project cycling and delivery robust and fast and keeps the entire developer team focused and centralized on product deliveries rather than discovering all possible needs and specifications and adding necessitated features.

Advantages of Prototyping Model

- **Better Requirement Understanding:** Helps in better understanding of user requirements, especially in cases where requirements are unclear at the start.
- **Improved User Involvement:** Early user feedback ensures that the final product meets user expectations.
- **Early Error Detection:** Design flaws, misunderstandings, or requirement gaps are identified early in the process.
- **Flexibility:** The iterative nature allows changes to be made easily as feedback is continuously integrated.

- **Enhances Customer Involvement and Satisfaction:** Prototypes enable you to test the user experience to refine the prototype and ensure that the final product meets the needs of your target audience. Involving customers and stakeholders in the prototyping process leads to higher satisfaction, as they can influence the product's direction from the start.
- **Reduced Development Costs:** Prototyping allows teams to identify potential issues early in the development process, reducing costs later on. By catching design flaws and functional gaps at an early stage, development teams can avoid expensive fixes during the more resource-intensive phases of development.

Disadvantages of the Prototyping Model

- **Scope Creep:** Frequent changes in the prototype can lead to uncontrolled growth in project scope.
- **Time-Consuming:** Continuous feedback and refinements may cause delays in the development process.
- **Inadequate Documentation:** Since the focus is more on the prototype, proper documentation may be overlooked, leading to issues later in the development.
- **User Confusion:** Users might mistake the prototype for a finished product, leading to unrealistic expectations. This misunderstanding can result in dissatisfaction if the final product doesn't match the prototype perfectly.

Disadvantages of the Prototyping Model

- **Costly** concerning time as well as money.
- There may be too much variation in requirements each time the prototype is evaluated by the customer.
- **Poor Documentation** due to continuously changing customer requirements.
- It is very difficult for developers to accommodate all the changes demanded by the customer.
- There is uncertainty in determining the number of iterations that would be required before the prototype is finally accepted by the customer.

Evolutionary Prototyping:

The evolutionary model is based on the concept of making an initial product and then evolving the software product over time with iterative and incremental approaches with proper feedback.

This model divides the development cycle into smaller, incremental waterfall models in which users can get access to the product at the end of each cycle. The development is carried out simultaneously with the feedback during the development.

Feedback is provided by the users on the product for the planning stage of the next cycle and the development team responds, often by changing the product, plan, or process. Therefore, the software product evolves with time.

All the models have the disadvantage that the duration of time from the start of the project to the delivery time of a solution is very high. The evolutionary model solves this problem with a different approach.

The evolutionary model suggests breaking down work into smaller chunks, prioritizing them, and then delivering those chunks to the customer one by one. The number of chunks is huge and is the number of deliveries made to the customer.

The main advantage is that the customer's confidence increases as he constantly gets quantifiable goods or services from the beginning of the project to verify and validate his requirements. The model allows for changing requirements as well as all work is broken down into maintainable work chunks.

Application of Evolutionary Model

- It is used in large projects where you can easily find modules for incremental implementation. Evolutionary model is commonly used when the customer wants to start using the core features instead of waiting for the full software.
- Evolutionary model is also used in object oriented software development because the system can be easily portioned into units in terms of objects.

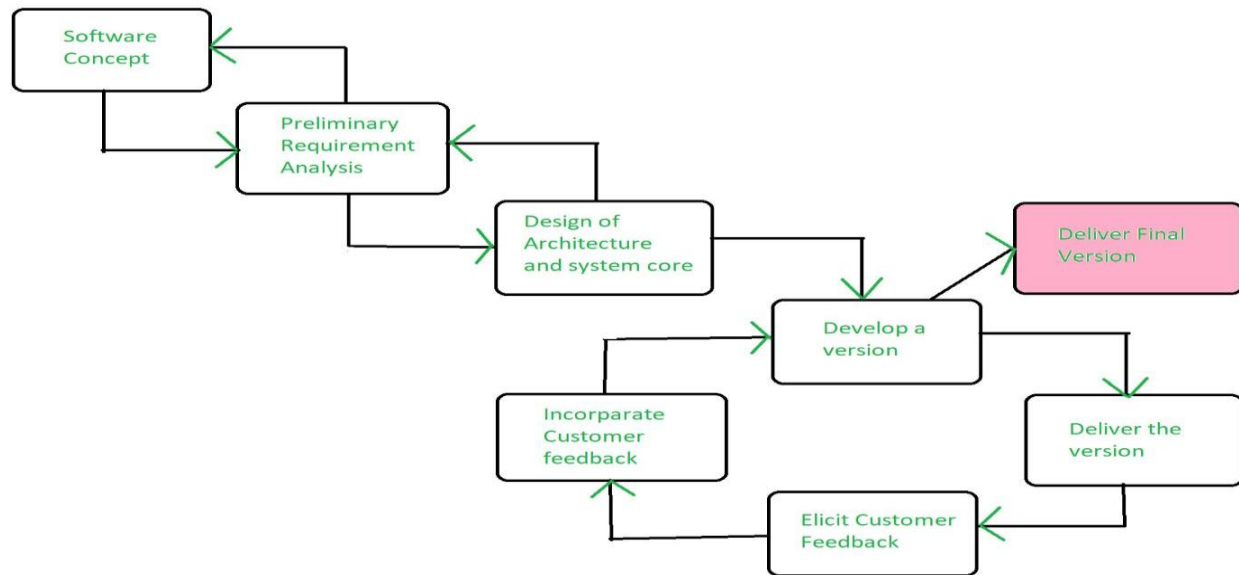


Figure1.4.3: Evolutionary Model

Necessary Conditions for Implementing this Model

1. Customer needs are clear and been explained in deep to the developer team.
2. There might be small changes required in separate parts but not a major change.
3. As it requires time, so there must be some time left for the market constraints.
4. Risk is high and continuous targets to achieve and report to customer repeatedly.
5. It is used when working on a technology is new and requires time to learn.

Advantages Evolutionary Model

- **Adaptability to Changing Requirements:** Evolutionary models work effectively in projects when the requirements are ambiguous or change often. They support adjustments and flexibility along the course of development.
- **Early and Gradual Distribution:** Functional components or prototypes can be delivered early thanks to incremental development. Faster user satisfaction and feedback may result from this.

- **User Commentary and Involvement:** Evolutionary models place a strong emphasis on ongoing user input and participation. This guarantees that the software offered closely matches the needs and expectations of the user.
- **Improved Handling of Difficult Projects:** Big, complex tasks can be effectively managed with the help of evolutionary models. The development process is made simpler by segmenting the project into smaller, easier-to-manage portions.

Disadvantages Evolutionary Model

- **Communication Difficulties:** Evolutionary models require constant cooperation and communication. The strategy may be less effective if there are gaps in communication or if team members are spread out geographically.
- **Dependence on an Expert Group:** A knowledgeable and experienced group that can quickly adjust to changes is needed for evolutionary models. Teams lacking experience may find it difficult to handle these model's dynamic nature.
- **Increasing Management Complexity:** Complexity can be introduced by organizing and managing several increments or iterations, particularly in large projects. In order to guarantee integration and synchronization, good project management is needed.
- **Greater Initial Expenditure:** As evolutionary models necessitate continual testing, user feedback and prototyping, they may come with a greater starting cost. This may be a problem for projects that have limited funding.

Spiral Model

Spiral model is a combination of both, iterative model and one of the SDLC model. It can be seen as if you choose one SDLC model and combine it with cyclic process (iterative model).

This model considers risk, which often goes un-noticed by most other models. The model starts with determining objectives and constraints of the software at the start of one iteration. Next phase is of prototyping the software. This includes risk analysis. Then one standard SDLC model is used to build the software. In the fourth phase of the plan of next iteration is prepared.

The spiral model has four phases. A software project repeatedly passes through these phases in iterations called Spirals.

- **Identification:** This phase starts with gathering the business requirements in the baseline spiral. In the subsequent spirals as the product matures, identification of system requirements, subsystem requirements and unit requirements are all done in this phase. This phase also includes understanding the system requirements by continuous communication between the customer and the system analyst. At the end of the spiral, the product is deployed in the identified market.

The following illustration is a representation of the Spiral Model, listing the activities in each phase.

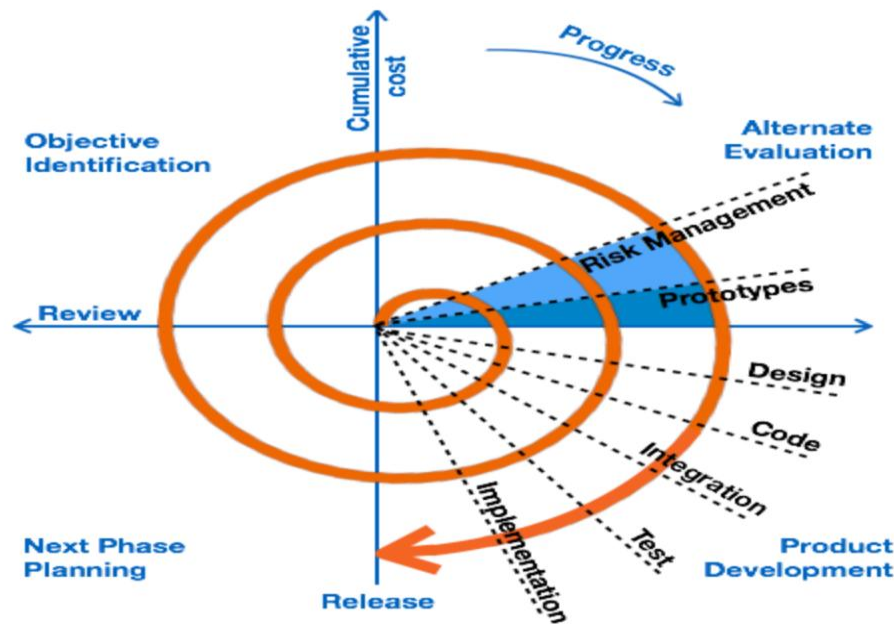


Figure1.4.4: Spiral Model

- **Design:** The Design phase starts with the conceptual design in the baseline spiral and involves architectural design, logical design of modules, physical product design and the final design in the subsequent spirals.
- **Construct or Build:** The Construct phase refers to production of the actual software product at every spiral. In the baseline spiral, when the product is just thought of and the design is being developed a POC (Proof of Concept) is developed in this phase to

get customer feedback. Then in the subsequent spirals with higher clarity on requirements and design details a working model of the software called build is produced with a version number. These builds are sent to the customer for feedback.

- **Evaluation and Risk Analysis:** Risk Analysis includes identifying, estimating and monitoring the technical feasibility and management risks, such as schedule slippage and cost overrun. After testing the build, at the end of first iteration, the customer evaluates the software and provides feedback.

The following explains the typical uses of a Spiral Model –

- When there is a budget constraint and risk evaluation is important.
- For medium to high-risk projects.
- Long-term project commitment because of potential changes to economic priorities as the requirements change with time.
- Customer is not sure of their requirements which is usually the case.
- Requirements are complex and need evaluation to get clarity.
- New product line which should be released in phases to get enough customer feedback.
- Significant changes are expected in the product during the development cycle.

Advantage and Disadvantages of Spiral Model:

The advantage of spiral lifecycle model is that it allows elements of the product to be added in, when they become available or known. This assures that there is no conflict with previous requirements and design.

This method is consistent with approaches that have multiple software builds and releases which allows making an orderly transition to a maintenance activity. Another positive aspect of this method is that the spiral model forces an early user involvement in the system development effort.

On the other side, it takes a very strict management to complete such products and there is a risk of running the spiral in an indefinite loop. So, the discipline of change and the extent of taking change requests is very important to develop and deploy the product successfully.

The advantages of the Spiral SDLC Model are as follows –

- Changing requirements can be accommodated.
- Allows extensive use of prototypes.
- Requirements can be captured more accurately.
- Users see the system early.
- Development can be divided into smaller parts and the risky parts can be developed earlier which helps in better risk management.

The disadvantages of the Spiral SDLC Model are as follows –

- Management is more complex.
- End of the project may not be known early.
- Not suitable for small or low risk projects and could be expensive for small projects.
- Process is complex
- Spiral may go on indefinitely.
- Large number of intermediate stages requires excessive documentation.

Unified Process Model:

The Unified Process (UP) is a software development process framework that is iterative and incremental, primarily focused on object-oriented design and development. The framework is also known as Rational Unified Process (RUP) and the Open Unified Process (Open UP). Some of the key features of this process include:

- It defines the order of phases.

- It is component-based, meaning a software system is built as a set of software components. There must be well-defined interfaces between the components for smooth communication.
- It follows an iterative, incremental, architecture-centric, and use-case driven approach.

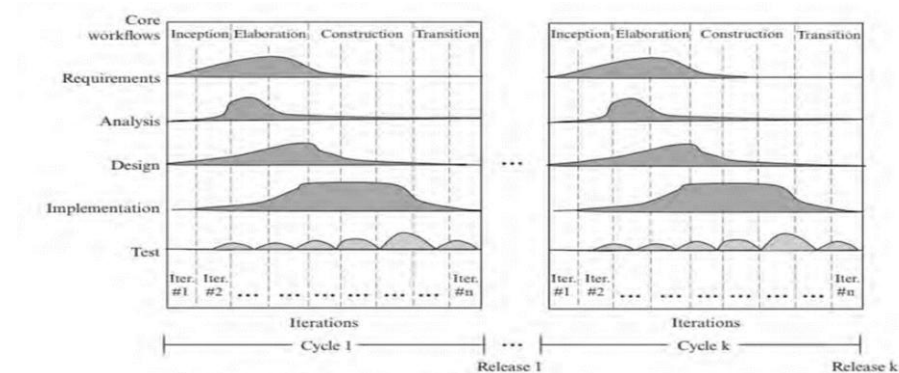


Figure 1.4.5: Unified Process Model

The Rational Unified Process or Unified Process (UP) consists of a series of cycles. Each cycle concludes with a release of the system. Each cycle has several iterations. The iterations are grouped into four phases: **Inception, Elaboration, Construction and Transition**. Each iteration goes through a series of workflow activities, including requirements analysis, design, implementation, and testing.

Inception: This phase produces a simplified use case model, a tentative software architecture, and a project plan. A use case models a business process of the application for which the system is being developed. The use case model contains the most critical use cases of the software system.

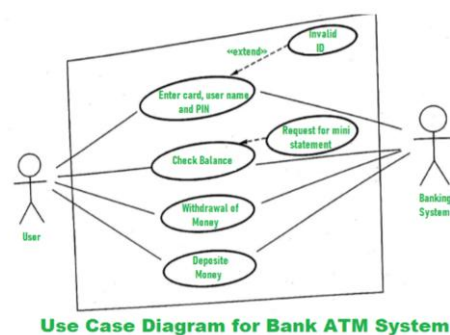


Figure 1.4.6: Use case diagram

Elaboration: During this phase, most use cases are specified, and UML diagrams representing the architectural design are produced. The most critical use cases of the software system are designed and implemented.

Construction: During the construction phase, the remaining use cases are iteratively developed and integrated into the system. The system can be incrementally installed in the target environment.

Transition: During the transition phase, activities are performed to deploy the software system. These include user training, beta testing, defect correction, and functional enhancement.

Features of UP:

- ***Use case driven*** :The UP focuses on identifying use cases and uses them to plan the iterations to develop and deploy the system.
- ***Architecture-centric*** : The UP determines the architecture or the overall structure of the system early in the life cycle, and uses it to guide the development of the software system
- ***Iterative and Incremental***: The system is developed and deployed iteratively and incrementally.

Agile Development Models:

In earlier days, the Iterative Waterfall Model was very popular for completing a project. But nowadays, developers face various problems while using it to develop software.

The main difficulties included handling customer change requests during project development and the high cost and time required to incorporate these changes. In the mid-1990s, the Agile Software Development Model was proposed to overcome these drawbacks of the Waterfall Model.

What is Agile Model?

Agile refers to something **quick or adaptable**. The Agile Model was primarily designed to help a ***project adapt quickly to change requests***. So, the main aim of the Agile model is to facilitate ***quick project completion***. Agile processes emphasize

- Teamwork,
- Joint application development with the users
- Design for change
- Rapid development and frequent delivery of small increments in short iterations.

The most significant aspect of the Agile model is determining the project scope, requirements, number, and duration of iterations at the start of the development process. Agile development is guided by agile values, principles, and best practices. The Agile Model refers to a group of development processes.

What is the Agile Manifesto?

The Agile Manifesto is a set of foundational principles for agile software development that emphasizes flexibility, collaboration, and customer satisfaction. It Outlines 4 core Values and Twelve Principles guide agile practices.

Four Core Values of the tackle Agile Manifesto

- Individuals and Interactions over Processes and Tools: Focuses on the importance of effective communication and collaboration among team members.
- Working Software over Comprehensive Documentation: Prioritizes the delivery of functional software as the primary measure of progress.
- Customer Collaboration over Contract Negotiation: Encourages customers and stakeholders to have active involvement throughout the development process.
- Responding to Change over Following a Plan: On changing requirements, embracing flexibility and ability to adapt even late in the development process.

Principles of Agile Manifesto: This Agile Manifesto is supported by 12 principles that help to implement the values.



Figure1.4.5(a): Principles of Agile Manifesto

1. Customer Satisfaction through Early and Continuous Delivery: This principle concentrates on the importance of customer satisfaction by providing information to customers early on time and also with consistency throughout the development process.
2. Welcome Changing Requirements, Even Late in Development: Agile processes tackle change for the customer's competitive advantage. Even late in development, changes in requirements are welcomed to ensure the delivered software meets the evolving requirements of the customer.
3. Deliver Working Software Frequently: This principle encourages the regular release of functional software increments in short iterations. This enables faster feedback and adaptation to changing requirements.
4. Collaboration between Business Stakeholders and Developers: This says the businesspeople and developers must work together daily throughout the project. There should be communication and collaboration between stakeholders and the development team regularly. This is crucial for understanding and prioritizing requirements effectively.
5. Build Projects around Motivated Individuals: This promotes in giving developers the environment and support they need and trusts them to complete the job successfully. Motivated and empowered individuals are more likely to produce work with quality and make valuable contributions to the project.

6. Face-to-face communication is the Most Effective: Face-to-face communication is the most effective method of discussion and conveying information. This principle depicts the importance of direct interaction which helps minimize misunderstandings, and hence effective communication is achieved.
7. Working Software is the Primary Measure of Progress: This principle emphasizes delivering functional and working software as the primary metric for project advancement. It encourages teams to prioritize the continuous delivery of valuable features, so it ensures that good progress is consistently achieved throughout the process. The primary goal is to provide customers with incremental value and also gather feedback early in the project life cycle.
8. Maintain a Sustainable Pace of Work: Agile promotes sustainable development. All people involved: The sponsors, developers, and users should be able to maintain a constant pace indefinitely. This principle depicts the need for a sustainable and consistent development pace. This helps in avoiding burnout and ensures long-term project success.
9. Continuous Attention to Technical Excellence and Good design: This principle is on the importance of maintaining high standards of technical craft and design, so it ensures the long-term ability in maintenance and adaptability of the software.
10. Simplicity—the Art of Maximizing the Amount of Work Not Done: Simplicity is essential. The objective here is to concentrate on the most valuable features and tasks and avoid unnecessary complexity as the art of maximizing the amount of work not done is crucial.
11. Self-Organizing Teams: Self-organizing teams provide the best architectures, requirements, and designs. These help in empowering teams to make decisions and organize to optimize efficiency and creativity.
12. Regular Reflection on Team Effectiveness: This makes the team reflect on how to become more effective at regular intervals and then adjust accordingly. Continuous improvement is very crucial for adapting to changing circumstances and optimizing the team's performance over time.

Steps in the Agile Model: The Agile Model is a combination of iterative and incremental process models. The phases involve in Agile (SDLC) Model are:

1. Requirement Gathering
2. Design the Requirements
3. Construction / Iteration
4. Testing / Quality Assurance
5. Deployment
6. Feedback

Agile Model Steps:

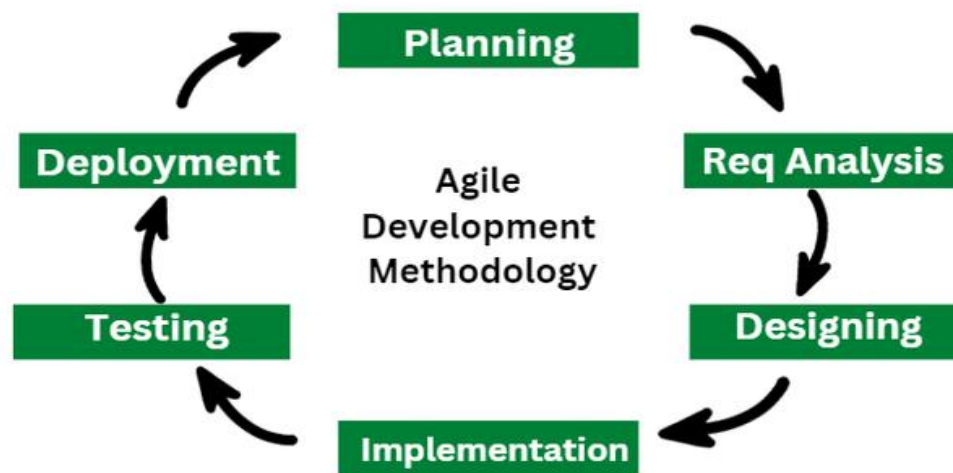


Figure 1.4.5(b): Agile Model life cycle

1. Requirement Gathering: In this step, the development team must gather the requirements, by interaction with the customer. development team should plan the time and effort needed to build the project. Based on this information you can evaluate technical and economical feasibility.

- Meet with the customer to really understand their needs and what they expect from the software.

- Identify the key requirements and business goals to make sure everyone is on the same page.
- Estimate how much time and effort it will take to develop the software.
- Assess if the project is technically possible and whether it's worth the investment from both a technical and economic standpoint.

2. Design the Requirements: In this step, the development team will use user-flow-diagram or high-level UML Diagrams to show the working of the new features and show how they will apply to the existing software. Wireframing and designing user interfaces are done in this phase.

- Designing the system: Once the requirements are gathered, the next step is to design the system's overall architecture based on those needs. This helps to verify the software is structured in a way that meets the user's expectations.
- Creating wireframes: Next, wireframes for the user interface (UI) are created. These are simple blueprints that show how the software will look and how users will interact with it, ensuring it's user-friendly and easy to navigate.
- High-level design with UML diagrams: At this stage, high-level designs using UML (Unified Modeling Language) diagrams are created to visually represent the software's structure and how different parts will work together.
- Prototyping for feedback: Prototypes are made to give stakeholders an early look at the software. This helps gather feedback early in the process and allows for adjustments before the full development begins.

3. Construction / Iteration: In this step, development team members start working on their project, which aims to deploy a working product. Each cycle typically consists between 1-4 weeks, and at the end, the team delivers a working version of the software.

- Development of Features: The team works on the features identified during the requirement and design phases.
- Coding and Implementation: New functionalities are coded and integrated into the software based on the goals for that specific iteration.

- Delivering a Working Product: After each iteration, a usable version of the software is ready.
- Incremental Improvement: With every cycle, the software is enhanced, adding more features and refining existing ones.

4. Testing / Quality Assurance: Testing involves Unit Testing, Integration Testing, and System Testing, Which help in the agile development models:

- Unit Testing: Unit testing is the process of checking small pieces of code to ensure that the individual parts of a program work properly on their own. Unit testing is used to test individual blocks (units) of code.
- Integration Testing: Integration testing is used to identify and resolve any issues that may arise when different units of the software are combined.
- System Testing: Goal is to ensure that the software meets the requirements of the users and that it works correctly in all possible scenarios.

5. Deployment: In this step, the development team will deploy the working project to end users. Once an iteration is finished and fully tested, the software is ready to be released to the end users. In Agile, deployment isn't a one-time event—it's an ongoing process. Updates and improvements are rolled out regularly, making sure the software keeps evolving and getting better with each release.

- Deploy the software to a test or live environment so that it can be used by customers or end-users.
- Make the software accessible to users, verifying they can start using it as expected.
- Verify the deployment goes smoothly and fix any issues that come up quickly

6. Feedback: This is the last step of the Agile Model. In this, the team receives feedback about the product and works on correcting bugs based on feedback provided by the customer.

- Take feedback from customers, users, and stakeholders after each iteration.
- Understand how well the product meets user needs and identify areas for improvement.
- Check for bugs or issues that need fixing.

- Make adjustments to the development plan based on feedback to improve the product further.

Agile Methods: Agile is a flexible framework with several approaches, each suited for different project needs. Here are some of the most common Agile methodologies like Kanban, Dynamic Systems Development Method, Scrum, Feature Driven Development, Extreme Programming and Extreme Project Management (XPM)

Advantages of Agile model:

- Customer satisfaction by rapid, continuous delivery of useful software.
- People and interactions are emphasized rather than process and tools. Customers, developers and testers constantly interact with each other.
- Working software is delivered frequently (weeks rather than months).
- Face-to-face conversation is the best form of communication.
- Close, daily cooperation between business people and developers.
- Continuous attention to technical excellence and good design.
- Regular adaptation to changing circumstances.
- Even late changes in requirements are welcomed

Disadvantages of Agile model:

- In case of some software deliverables, especially the large ones, it is difficult to assess the effort required at the beginning of the software development life cycle.
- There is lack of emphasis on necessary designing and documentation.
- The project can easily get taken off track if the customer representative is not clear what final outcome that they want.
- Only senior programmers are capable of taking the kind of decisions required during the development process. Hence it has no place for newbie programmers, unless combined with experienced resources.

1.5 WHAT ARE SOFTWARE DEVELOPMENT METHODOLOGIES:

Software Process: A process in general specifies only the activities and how they relate to each other. It does not specify how to carry out the activities.

Software Development Methodology: A software Methodology defines the steps or how to carry out the activities of a software process. The definition means that a methodology is an implementation of a process. Software development needs a process and a methodology.

Difference between Process and Methodology

Process	Methodology
High-level stages or steps in software development.	Specific approach or framework to execute a process.
What needs to be done (the stages of development).	How it is done (techniques, tools, and practices).
Ensures that development follows a structured approach.	Guides teams on specific best practices for execution.
Broad, covering the overall development life cycle.	Detailed, focused on daily activities and execution.
Example: Waterfall, Agile, Spiral, V-Model.	Example: Scrum, Extreme Programming (XP), Kanban.

Benefits of a Methodology:

- Enables the development team to ***focus on the important tasks***, and ***specifies how to perform these tasks*** correctly to produce the desired software system.
- ***Improves communication and collaboration.***
- Improves ***design quality*** and ***software productivity***
- Forms the basis for ***process improvement*** because measurements of software quality, productivity, cost, and time to market can be defined and applied to identify strengths and weaknesses.

- Forms the basis for ***process automation*** because many of the methodological steps can be mechanically carried out, making software automation much easier.
- A methodology that is ***easy to learn*** and ***use*** enables beginners to produce quality software.

Different Types of Software Development Methodologies:

- Structured Analysis and Structured Design (SA/SD)
- Classical OO Methodologies
- Agile Methods

1. Structured Analysis and Structured Design (SA/SD):

1. Popular in the 1970s and 1980s. Based on the principle of ***structured programming***, breaking down a software system into smaller, more manageable components.

In SA/SD, the software development process is divided into two phases:

- Structured Analysis
- Structured Design

Structured Analysis:

1. Focuses on understanding and documenting the requirements of a system
2. The ***primary objective is to create a clear and precise representation of the system's functionalities and its interactions with the external environment.***
3. ***Uses data flow diagrams (DFDs)*** to model the business processes of real-world applications
4. DFDs represent the flow of data within the system and how it is processed. They help visualize the data transformations and identify the input and output sources.

The steps of structured analysis:

1. Begin with the construction of a top-level DFD, called the context diagram.

2. Repeatedly decompose complex processes into simpler processes. This continues until the leaf-level processes can be easily implemented.
3. The input, output, and their data structures, as well as the algorithms of the processes are specified.

Structured Design:

- Once the requirements have been thoroughly analysed and documented, the focus shifts to Structured Design, where the ***goal is to transform the requirements into a structured and modular software architecture.***
- Structured Design promotes a systematic and organized approach to building software systems, ***emphasizing clarity, maintainability, and ease of understanding.***

2. Classical OO Methodologies

- Object-oriented methodologies ***focus on using objects*** (which encapsulate data and behavior) to model and design systems. These methodologies are ***aimed at building modular, reusable, and maintainable software systems.***

Key Features of Object-Oriented Methodologies:

1. **Encapsulation:** Wrapping data (attributes) and behavior (methods) together into objects.
2. **Abstraction:** Hiding the internal details and showing only the necessary functionality.
3. **Inheritance:** Allowing objects or classes to inherit properties and behaviors from other objects/classes.
4. **Polymorphism:** Allowing objects to be treated as instances of their parent class, providing flexibility in how methods are implemented.

Types of OO Methodologies:

There are 3 widely known classical OO methodologies. They are:

- The Booch Method
- The Object Modelling Technique (OMT)
- The Use case driven approach.

1. The Booch Method:

- The Booch Methodology is an approach to software development, particularly in Object-Oriented Analysis and Design (OOAD), pioneered by Grady Booch.
- ***Framework for designing and developing software systems using object-oriented principles.***
- Key components of the Booch Methodology include **object modelling, class diagrams, dynamic modeling and architectural design.**
- It emphasizes ***modularity, encapsulation, and abstraction*** to create flexible and maintainable software solutions.

2. The Object Modelling Technique (OMT):

The Object Modelling Technique (OMT) is a ***method for analyzing and designing object-oriented systems.*** Developed by James Rumbaugh in the early 1990s, OMT provides a framework for modeling and organizing the elements of a system in a way that promotes clarity and efficiency in design.

Phases of OMT:

- Object Modeling: Describes the static structure of the system
- Dynamic Modeling: Focuses on the behavior of the system over time.
- Functional Modeling: Concentrates on the system's functions and their interactions

Advantages of OMT:

- **Structured Approach:** Provides a clear and structured methodology for object-oriented analysis and design.

- **Reusability:** Promotes reuse of classes and objects, enhancing system maintainability and flexibility.
 - **Visualization:** Uses visual models to represent complex systems, making it easier to understand and communicate system design.
- 3. The Use-case driven approach:**
- Focuses on *defining and analyzing a system's requirements through its interactions with external actors*.
 - This approach emphasizes *capturing functional requirements by describing how users (or other systems) will interact with the system* to achieve specific goals.

Key Concepts: Use Cases, Actors, Scenarios and System Boundaries

Steps in the Use Case Driven Approach

- Identify Actors
- Define Use Cases
- Create Use Case Diagrams
- Detail Use Cases
- Refine and Validate
- Integrate with Design and Development

Benefits of the Use Case Driven Approach

- **User-Centric:** Focuses on how users interact with the system, ensuring that the system meets their needs and expectations.
- **Clarity:** Provides a clear and understandable description of system functionality, making it easier to communicate requirements with stakeholders.
- **Documentation:** Serves as a valuable documentation tool throughout the development lifecycle, including requirements analysis, design, and testing.
- **Scope Management:** Helps in defining the scope of the system by identifying which features and functions are included based on user interactions.

What are Software Development Methodologies?

Software Development Methodologies are defined as a process in which the software developers design, develop and test the new computer programs and it is also used in the betterment of the software projects. These software development methodologies follow a particular design philosophy in which it helps the software developers to align these processes and the features of the software product. With the help of these agile methodologies also simplify the tasks and improve the collaboration in the companies.

The main objective of Software Development companies is to provide high quality software products at low cost. Therefore, proper planning is required and proper management is required for the software development process. Thus a proper methodology is important for achieving this type of objectives by the companies and organizations. There are multiple types of Software development methodologies and in this article a detailed knowledge is provided about the Top 15 Software Development Methodologies.

1. Agile Methodology: Kanban
2. Scrum Methodology
3. Extreme Programming (XP)
4. Behavior-Driven Development
5. Lean Development
6. Waterfall Methodology
7. Feature Driven Development (FDD)
8. Spiral Methodology
9. Rapid Application Development (RAD)
10. Prototyping Methodology
11. Rational Unified Process Methodology
12. Adaptive Software Development
13. Dynamic Systems Development Model
14. DevOps Methodology
15. Joint Application Development Methodology

Agile Methodology

In the IT field Agile methodology is one of the most popular software engineering techniques in which various software production methodologies are related to the principles of agile. The main objective of Agile methodologies is to finish the product with collaborative efforts and the main benefit of this methodology is that it ensures regular release of products and continuous improvement with every iteration.

Advantages of Agile Methodology

- Due to involvement of small iteration it delivers high quality of output.
- With the help of Agile methodology allows creative improvements whenever working on the software product.
- The agile methodology is popular for its minimal reliance and adaptivity on the initial documentation.

Disadvantages of Agile Methodology

- Agile methodology doesn't consist of any deadlines.
- Agile methodology also lacks clarity and the project vision.

Agile methods are a group of software development methodologies based on the principles of flexibility, collaboration, and customer-centric development. It aim to deliver software incrementally, with frequent releases that provide value to the customer early in the development process.

The most widely-used Agile methodologies:

1. Scrum
2. Kanban
3. Extreme Programming (XP)

Scrum Methodology: Scrum is one of the most popular frameworks which is based on the agile methodology which is empirical in nature and it is famous for managing projects which do not have well defined feedback from the customers.

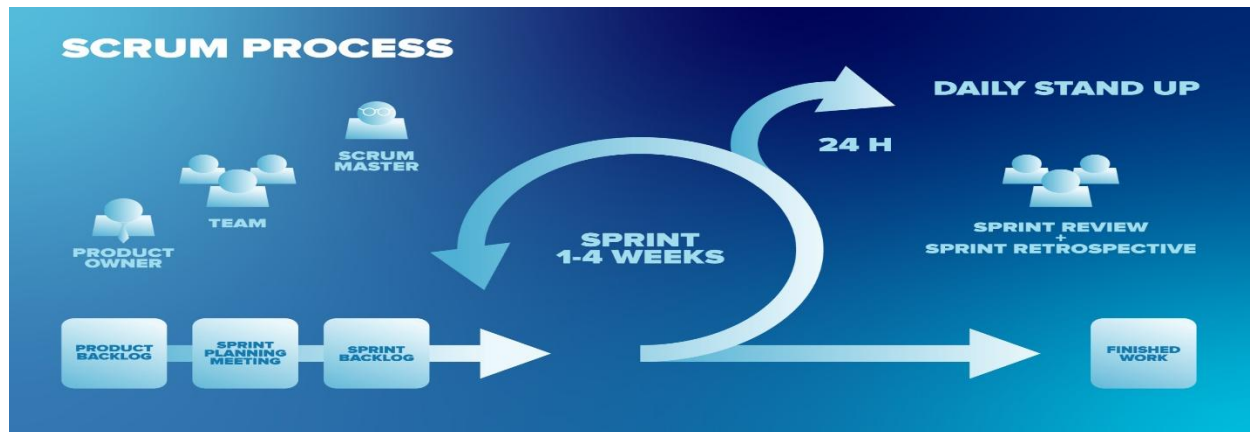


Figure1.5.1: Scrum Process

Key Concepts:

- Work is done in short, iterative cycles called sprints (typically 2-4 weeks).
- A Scrum team consists of a Product Owner (who represents the customer), a Scrum Master (who facilitates the process), and the development team.
- At the end of each sprint, a potentially shippable product increment is delivered.

Key Events:

- Sprint Planning: Define the goal for the sprint.
- Daily Standups: Quick daily meetings to track progress.
- Sprint Review: Review the completed work at the end of the sprint.
- Sprint Retrospective: Reflect on what went well and what can be improved.

Example: A team working on an e-commerce platform may complete features like user registration or product search within a sprint, and the features are tested and ready for release by the end of that sprint.

Advantages of Scrum Methodology

- Scrum methodology helps the team members make the decisions on the main principal project.
- With the help of the scrum methodology the developers can detect the problems fastly and easily.

Disadvantages of Scrum Methodology

- Scrum is not effective for the junior team members and it is also less effective for the big project types.

Kanban: is an approach to evolutionary and incremental systems and process change for organizations. A work-in-progress limited pull system is the central mechanism to uncover system operation (or process) complications and encourage collaboration to continuously improve the system.

Kanban is best suited in the below scenarios:

- Dynamic/ frequent changing requirements which need to be delivered faster.
- In case of changing priorities, the team can pull the prioritized work as soon as the WIP limit drops.
- Frequent releases are there (Periodically).
- When incoming work is continuous.

Key Concepts:

- Focuses on visualizing workflow using a Kanban board to track work progress.
- Work is broken into smaller tasks, each moving through stages such as To Do, In Progress, and Done.
- Limits are set on how many tasks can be worked on at any one time to avoid overloading the team.

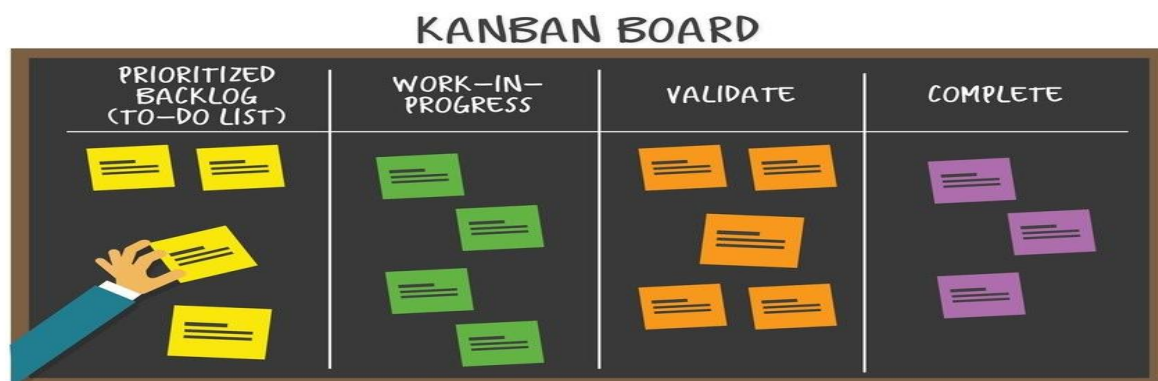


Figure 1.5.2: Kanban Board

Continuous delivery of work, no fixed-length iterations.

Example: A software team uses a Kanban board to track and limit the number of tasks being worked on, ensuring they complete them efficiently without multitasking.

Advantage of Kanban Method:

- Flexibility – Tasks can be reprioritized anytime without disrupting the workflow.
- Visual Workflow – Easy to track progress using Kanban boards (To Do → In Progress → Done).
- Improved Efficiency – Limits Work in Progress (WIP), preventing team overload.
- Faster Delivery – Continuous delivery approach reduces cycle time. Better Collaboration – Promotes transparency, making it easy for team members and stakeholders to understand the status.
- Waste Reduction – Focuses on completing tasks before starting new ones, reducing bottlenecks.

Disadvantage of Kanban Method:

- Lack of Timeframes – No fixed time-box (like in Scrum sprints), which may delay delivery deadlines.
- Risk of Scope Creep – Continuous addition of new tasks can make it hard to control project scope.
- Not Ideal for Large Teams – Works best with small teams; scaling it up can cause complexity.
- Requires Discipline – Teams must strictly follow WIP limits; otherwise, the system loses effectiveness.
- Limited Predictability – Harder to forecast release dates since there are no fixed iterations.
- May Overlook Long-Term Planning – Strong focus on daily tasks can sometimes ignore broader project goals.

Extreme Programming (XP)

XP or Extreme programming is also used to define the agile methodology whose main objective is to develop a fully functional product as it is also helpful in developing complex projects with fixed deadlines. XP is mostly suitable for developing software in unstable environments.

Key Concepts:

- Focuses heavily on technical excellence and frequent releases.
- Practices include pair programming (two developers working together on one task), test-driven development (TDD) (writing tests before coding), and continuous integration (frequent merging of code into a shared repository).
- Emphasizes customer involvement through continuous feedback.

Example: A team working on a mobile app uses XP practices like writing automated tests for each feature and deploying code to production multiple times a day for immediate feedback.

Advantages of Extreme Programming (XP)

- Extreme programming is cost effective and it works well with large and small teams.
- XP is also useful for risk management which overall increases the chance of success.

Disadvantages of Extreme Programming (XP)

- Extreme programming needs regular reviews and meetings between the stakeholders which leads to more time consumption.

MODULE 1 – MODEL QUESTION

1. What is software process and project.
2. Explain about component software process.
3. What is SDLC. What are different SDLC models.
4. Describe classical waterfall model and iterative development model of Software development. Draw appropriate diagrams. Compare the two models.
5. Discuss about prototyping model. Explain its merits and Demerits.
6. Discuss in detail about the layers involved in Unified Process model.
7. Discuss in detail about Time Boxing model.
8. Explain different phases of Extreme Programming (XP) with a neat diagram .

[or]

What are the phases of extreme programming? Explain.

[or]

Describe extreme programming process.

9. What is Agility? What is an agile process?
10. What is Agility? Explain Extreme programming Values.22
11. What are generic process models and explain process models?[Note : Generic process Model = WaterFall Model and explain all other models for process models.]
12. Explain about project management process.
13. What are perspective process models.

[There are two types of prescriptive process models. They are:

1. The Waterfall Model

2. Incremental Process model]

NOTE: when question is asked like this explain about waterfall and incremental.]

14. Explain about agile model for software development. Compare it with waterfall model, prototyping model and iterative development model.

15. Why does an iterative process make it easier to manage change? Is it possible to complete a project in just one iteration and still be agile?

Explain your answers.

16. Describe how incremental process models are better than water fall model.

17. What process models will you use in various projects.